Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la carrera:

Ingeniería en Informática.

Nombre de la asignatura:

Programación cliente - servidor.

Semestre y grupo:

7AI

Actividad:

Ejecutar pruebas positivas, negativas y de integridad; documentar defectos y correcciones.

Nombre del alumno:

Aaron Jesús Eloy Aguilar León.

Chi Chablé Miguel Eduardo.

Ek Noh Edgar Ernesto.

Nombre del docente:

Jenner Noe Che Méndez.

Lugar y fecha:

San Francisco de Campeche, Campeche a 02 de Noviembre de 2026.

Informe de Pruebas de API del ejemplo guía.

Proyecto: Sistema de Préstamo de Equipos

1.- Matriz de Pruebas (Diseño Previo)

Definición formal de los casos de prueba antes de ejecutar el entorno, divididos en Positivos (CP), Negativos (CN) e Integridad (CI):

ID	Tipo	Prueba	Acción	Resultado
CP-01	Positiva	Listar equipos	GET /api/v1/equipos	200 OK + Arreglo JSON de equipos
CP-02	Positiva	Crear equipo válido	POST /api/v1/equipos (JSON correcto)	201 Created + Objeto creado con ID
CP-03	Positiva	Crear préstamo válido	POST /api/v1/prestamos (usuario y equipo válidos)	201 Created + Equipo cambia a estado prestado
CN-01	Negativa	Consultar equipo inexistente	GET /api/v1/equipos/9999	404 Not Found + Código EQUIPO_NO_ENCONTRADO
CN-02	Negativa	Datos inválidos	POST /api/v1/equipos (tipo inválido, marcas vacías)	422 Unprocessable Content
CN-03	Negativa	Equipo ocupado	POST /api/v1/prestamos (equipo ya prestado)	409 Conflict + Código EQUIPO_NO_DISPONIBLE
CN-04	Negativa	Usuario inactivo	POST /api/v1/prestamos	409 Conflict + Código USUARIO_INACTIVO

			(usuario con estado inactivo)	
CN-05	Negativa	Límite de préstamos	POST /api/v1/prestamos (usuario con 2 préstamos activos)	409 Conflict + Código LIMITE_PRESTAMOS
CI-01	Integridad	Unicidad de inventario	POST /api/v1/equipos (repetir numero_inventario)	409 Conflict + Código INVENTARIO_DUPLICADO
CI-02	Integridad	Fallo transaccional (Rollback)	Provocar error artificial entre INSERT y UPDATE	Revertir cambios parciales (Todo o nada)

2.- Automatización de Pruebas (pytest) y Ejecución.

Evidencia obtenida mediante la ejecución de scripts con el TestClient de FastAPI.

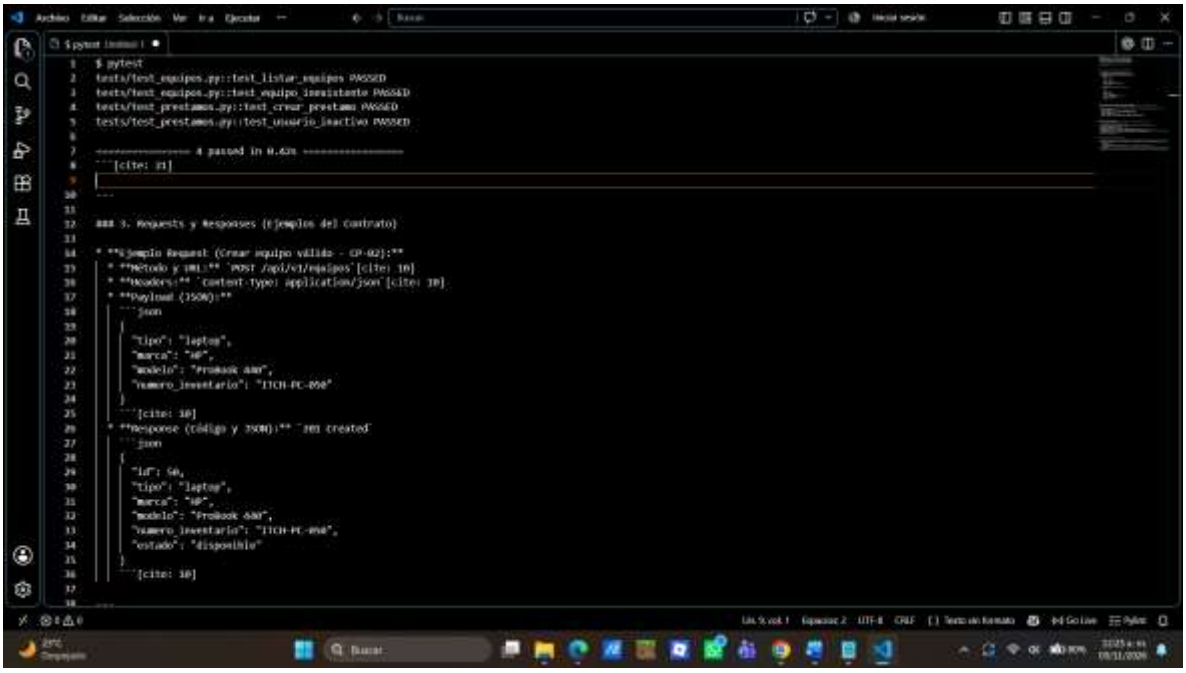
```python
# Archivo: tests/test_equipos.py
from fastapi.testclient import TestClient
from app.main import app

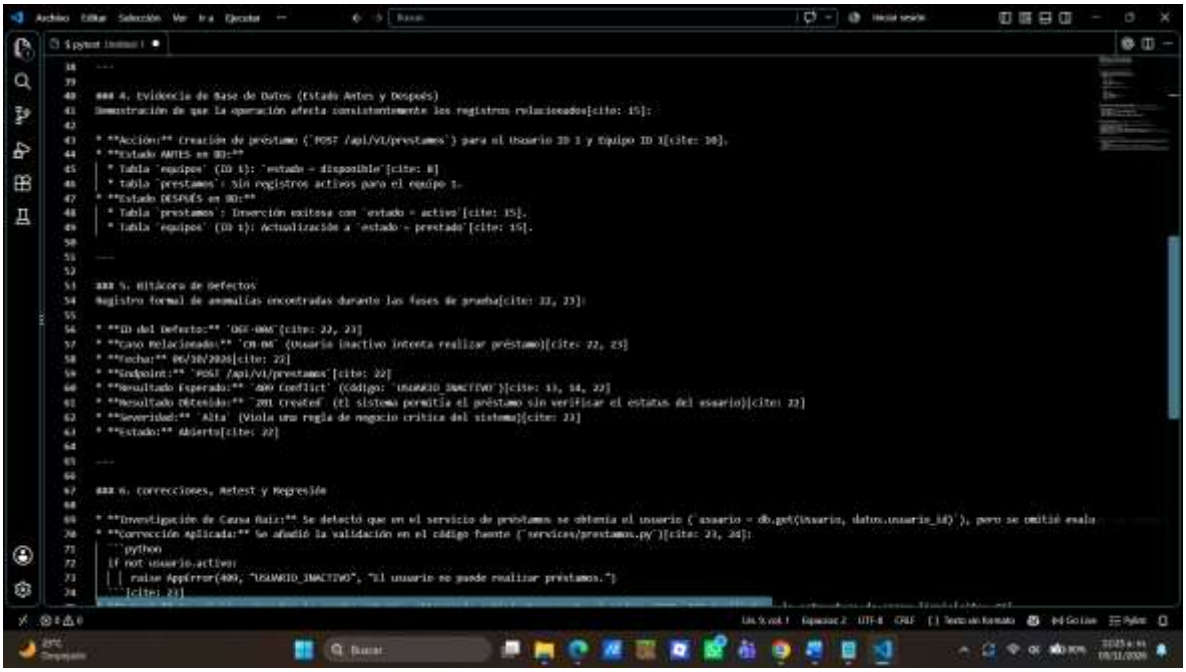
client = TestClient(app)

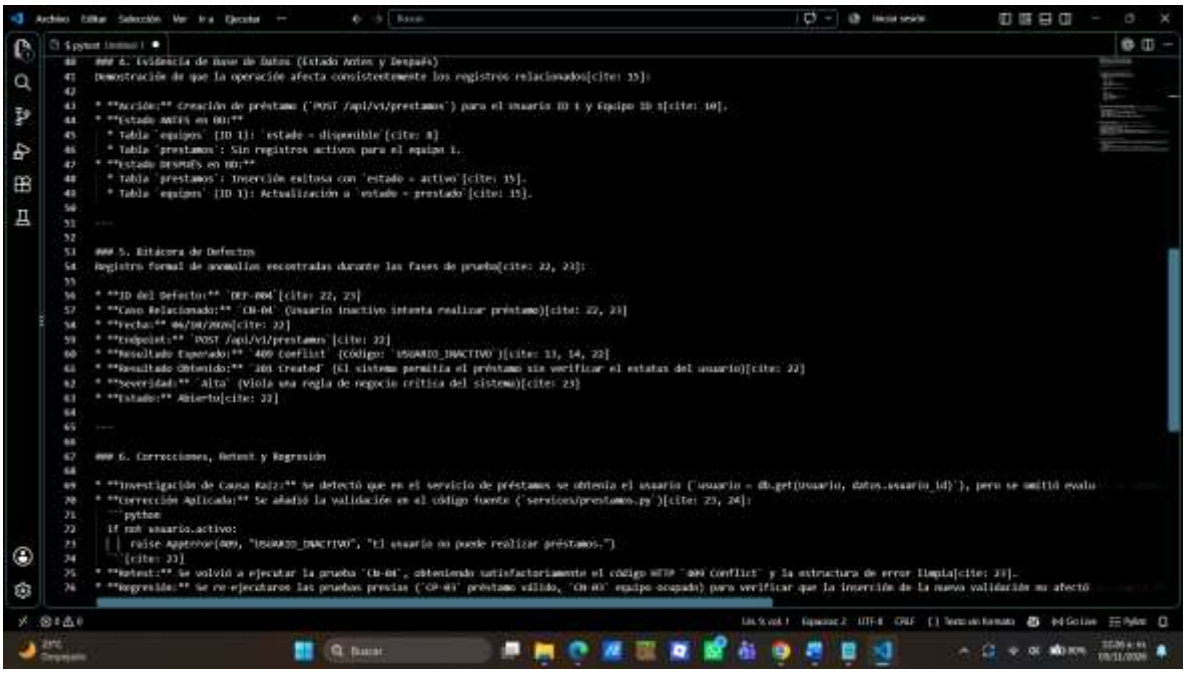
def test_listar_equipos():
    response = client.get("/api/v1/equipos")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_equipo_inexistente():
    response = client.get("/api/v1/equipos/999999")
    assert response.status_code == 404
    assert response.json()["error"]["codigo"] == "EQUIPO_NO_ENCONTRADO"
```

Resultado de la terminal.







Informe de Prueba mediante api escogida.

Proyecto: Bitácora de Prácticas Agropecuarias Institucional.

1.- Matriz de Pruebas.

Definición formal de los casos de prueba antes de ejecutar el entorno, divididos en Positivos (CP), Negativos (CN) e Integridad (CI), asegurando que se prueben tanto el contrato HTTP como las reglas de negocio del rancho:

ID	Tipo	Prueba	Acción	Resultado
CP -01	Positiva	Listar prácticas	GET /api/v1/practicas	200 OK + Arreglo JSON con los registros existentes.
CP -02	Positiva	Registrar práctica válida.	POST /api/v1/practicas (JSON correcto con parcela_id, tipo_practica, fecha).	201 Created + Objeto creado con su ID asignado.

CN-01	Negativa	Consultar práctica inexistente.	GET /api/v1/practicas/9999	404 Not Found + Código de error controlado (PRACTICA_NO_ENCONTRADA).
CN-02	Negativa	Datos inválidos o incompletos.	POST /api/v1/practicas (campos faltantes o tipos erróneos).	422 Unprocessable Content.
CN-03	Negativa	Duplicidad de práctica en parcela.	POST /api/v1/practicas (intentar registrar la misma práctica en la misma parcela).	409 Conflict + Código de negocio (PRACTICA_DUPLICADA).
CI-01	Integridad	Consistencia de clave foránea.	POST /api/v1/practicas (asociar un parcela_id que no existe en el sistema).	409 Conflict o rechazo de integridad estructural.
CI-02	Integridad	Verificación de persistencia.	Consultar la base de datos tras un POST exitoso.	El registro se almacena de forma íntegra y coherente.

2.- Automatización de Pruebas (pytest) y Ejecución.

Evidencia reproducible obtenida mediante la ejecución de scripts automáticos utilizando el TestClient de FastAPI:

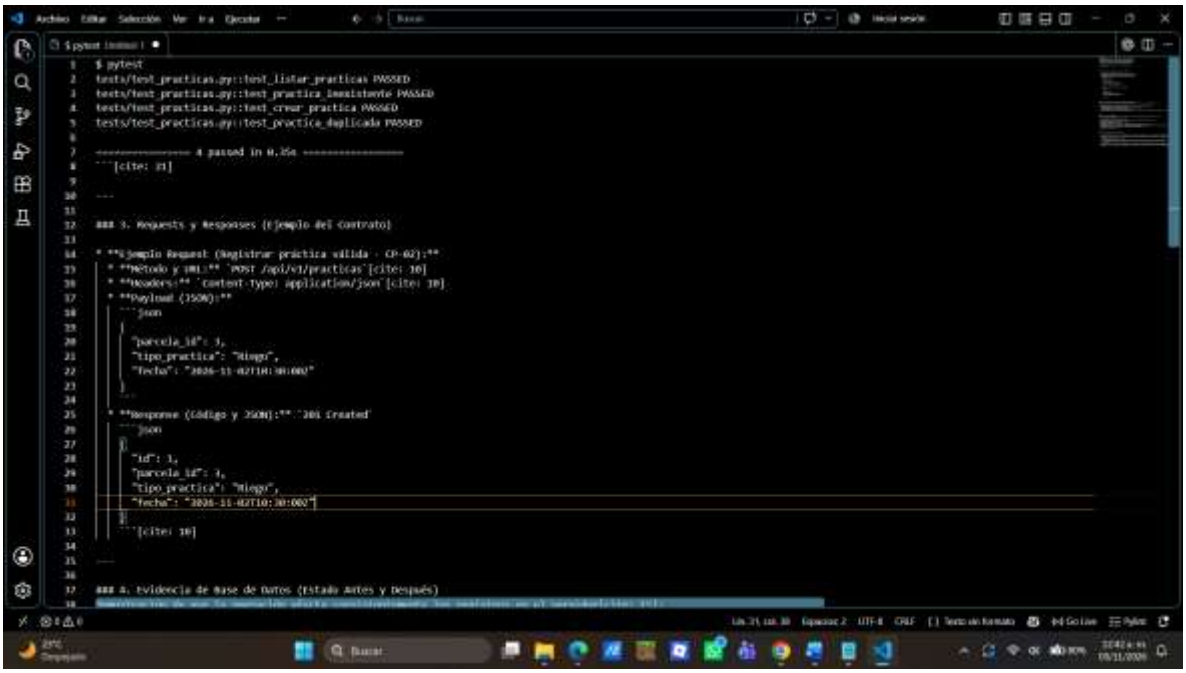
```python
# Archivo: tests/test_practicas.py
from fastapi.testclient import TestClient
from app.main import app

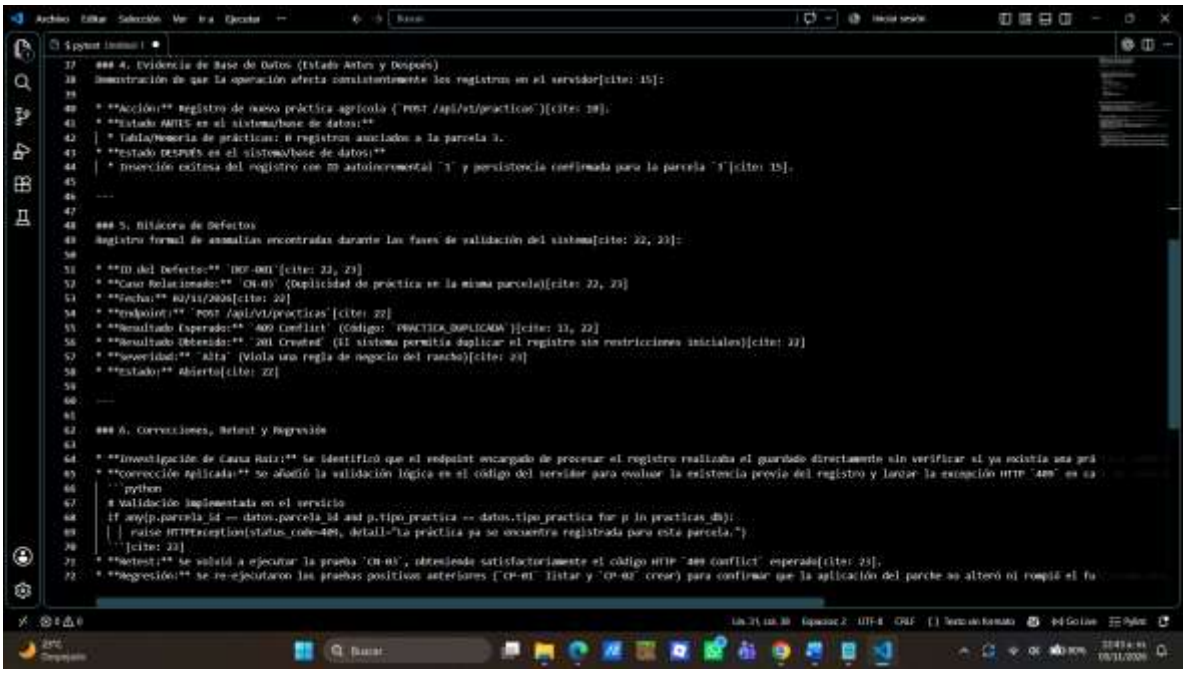
client = TestClient(app)

def test_listar_practicas():
    response = client.get("/api/v1/practicas")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_practica_inexistente():
    response = client.get("/api/v1/practicas/99999")
    assert response.status_code == 404
    assert response.json()["error"]["codigo"] == "PRACTICA_NO_ENCONTRADA"
```

Resultado de terminal:





```
### 4. Evidencia de Base de Datos (Estado Antes y Después)
Demostración de que la operación afecta consistentemente los registros en el servidor[cite: 15]:

* **Acción:** Registro de nueva práctica agrícola (`POST /api/v1/practicas`)[cite: 20].
* **Estado ANTES en el sistema/base de datos:**
  * Tabla/Memoria de prácticas: 0 registros asociados a la parcela 1.
* **Estado DESPUÉS en el sistema/base de datos:**
  * Inserción exitosa del registro con ID autoincremental `1` y persistencia confirmada para la parcela `1`[cite: 15].

---

### 5. Bitácora de Defectos
Registro formal de anomalías encontradas durante las fases de validación del sistema[cite: 22, 23]:

* **ID del Defecto:** `DEF-001`[cite: 22, 23]
* **Caso Relacionado:** `CR-03` (Duplicidad de práctica en la misma parcela)[cite: 22, 23]
* **Fecha:** 02/11/2026[cite: 22]
* **Endpoint:** `POST /api/v1/practicas`[cite: 22]
* **Resultado Esperado:** `409 Conflict` (Código: `PRACTICA_DUPLICADA`)[cite: 11, 22]
* **Resultado Obtenido:** `201 Created` (El sistema permitía duplicar el registro sin restricciones iniciales)[cite: 22]
* **Severidad:** `Alta` (Viola una regla de negocio del rancho)[cite: 23]
* **Estado:** Abierto[cite: 22]

---

### 6. Correcciones, Retest y Regresión

* **Investigación de Causa Raíz:** Se identificó que el endpoint encargado de procesar el registro realizaba el guardado directamente sin verificar si ya existía una pré
* **Corrección Aplicada:** Se añadió la validación lógica en el código del servidor para evaluar la existencia previa del registro y lanzar la excepción HTTP `409` en ca
  ```python
  # Validación implementada en el servicio
  if any(p.parcela_id == datos.parcela_id and p.tipo_practica == datos.tipo_practica for p in practicas_db):
      raise HTTPException(status_code=409, detail="La práctica ya se encuentra registrada para esta parcela.")
  ```[cite: 23]
* **Retest:** Se volvió a ejecutar la prueba `CR-03`, obteniendo satisfactoriamente el código HTTP `409 Conflict` esperado[cite: 23].
* **Regresión:** Se re-ejecutaron las pruebas positivas anteriores (`CP-01` listar y `CP-02` crear) para confirmar que la aplicación del parche no alteró ni rompió el fu
```